Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнила:
Студентка 4 курса
Группы ИИ-25
Мархель А.Д.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: на основе многослойного персептрона p-N-1 решить задачу прогнозирования временного ряда. Научиться формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Ход работы:

1. Сгенерировать ряд по формуле варианта и построить его график с границей обучающего и тестового участков.
2. Нормализовать ряд по минимуму и максимуму обучающего участка, использовать обратный пересчёт для метрик.
3. Сформировать обучающую и тестовую выборки методом скользящего окна размера p.
4. Реализовать МСП p-N-1: скрытый слой — сигмоида, выход — линейный, MSE, онлайн-обучение с перемешиванием.
5. Критерий остановки: $Es \le Ee = 10^{-4}$ (нормализованная шкала) или 20 000 эпох; подобрать η из 0.01-0.3.
6. Базовый эксперимент с p и N варианта: эпохи, Es, MAE, RMSE, sMAPE на обучении и тесте.
7. Исследование окна: $p \in \{2, 4, 6, 12\}$ при $N = 4$, по 5 запусков.
8. Исследование скрытого слоя: $N \in \{1, 2, 4, 8\}$ при $p = 6$, по 5 запусков.
9. Рекурсивный прогноз на 30 шагов от $k = 140$ для лучшей конфигурации.
10. Построить графики: ряд с прогнозом, сходимость, «эталон - прогноз», рекурсивный прогноз и рост ошибки.
11. Реализовать режим функционирования и продемонстрировать его на 8 окнах.
12. Выполнить рукописную часть (схема пайплайна, ручной расчёт шага обучения) и сформулировать вывод.

Вариант 6

$$x(k) = a \cdot \cos(b \cdot k) + c \cdot \sin(d \cdot k), \quad k = 0, 1, \ldots, 199$$

Вариант	a	b	c	d	p	N
6	1.2	0.28	0.6	0.65	6	4

Обучающий участок — $k = 0 \ldots 139$ (140 точек, 70 %), тестовый — $k = 140 \ldots 199$ (60 точек, 30 %). Сеть по p предыдущим значениям предсказывает следующее: $x(k) = F(x(k-1), x(k-2), \ldots, x(k-p))$.

Описание метода

1) Нормализация. Параметры берутся только по обучающему участку: x_min = −1.7840, x_max = 1.7830 (весь ряд: −1.7958 … 1.7989).

$$x_norm = (x - x_min) \,/\, (x_max - x_min)$$

$$x = x_norm \cdot (x_max - x_min) + x_min$$

Обратный пересчёт применяется ко всем прогнозам перед расчётом метрик (MAE, RMSE, sMAPE — в исходной шкале). Min и max нельзя брать по всему ряду: тестовый участок моделирует будущее, неизвестное при обучении. Использование его значений — утечка данных, которая завышает оценку качества и невозможна при реальном прогнозировании. На нашем ряду тест в нормализованной шкале занимает [−0.0033; 1.0045], т.е. немного выходит за [0; 1].

2) Скользящее окно. Пример для момента k: вход (x(k−p), …, x(k−1)) — p значений по времени, цель — x(k). Обучающие цели k = p…139, тестовые — k = 140…199 (окна тестовых примеров могут начинаться в обучающем участке: это уже известная история, а прогнозируемые значения — только тестовые). Число примеров:

p	2	4	6	12
обучающих	138	136	134	128
тестовых	60	60	60	60

3) Архитектура p-N-1. Входной слой — p значений окна, скрытый слой — N сигмоидных нейронов, выход — один линейный нейрон:

$$S_j = \Sigma_i \, w_ij \cdot x_i + b_j, \quad h_j = \sigma(S_j) = 1/(1 + e\text{^}(-S_j))$$

$$y = \Sigma_j \, v_j \cdot h_j + b_out$$

4) Вывод градиентов. Ошибка на одном примере E = ½(d − y)². Для линейного выхода $\partial y/\partial s = 1$, поэтому

$$\delta_out = -\partial E/\partial s_out = (d - y) \cdot 1 = d - y$$

$$\partial E/\partial v_j = -\delta_out \cdot h_j, \quad \partial E/\partial b_out = -\delta_out$$

Для скрытого нейрона ошибка передаётся через вес v_j и умножается на производную сигмоиды σ′(S) = h(1 − h):

$$\delta_j = -\partial E/\partial S_j = \delta_out \cdot v_j \cdot h_j \cdot (1 - h_j)$$

$$\partial E/\partial w_ij = -\delta_j \cdot x_i, \quad \partial E/\partial b_j = -\delta_j$$

Обновление весов (онлайн, после каждого примера, градиентный спуск со скоростью η):

$$v_j \mathrel{+}= \eta \cdot \delta_out \cdot h_j, \;\; b_out \mathrel{+}= \eta \cdot \delta_out$$

$$w_{ij} \mathrel{+}= \eta \cdot \delta_j \cdot x_i, \quad b_j \mathrel{+}= \eta \cdot \delta_j$$

В отличие от ЛР №1-3 на выходе нет множителя $y(1 - y)$: линейный выход не насыщается. Порядок примеров перемешивается в каждой эпохе. После эпохи вычисляется средняя ошибка по всей обучающей выборке $Es = (1/L) \cdot \Sigma(d - y)^2$ (нормализованная шкала), обучение останавливается при $Es \le 10^{-4}$ или через 20 000 эпох. Веса и пороги инициализируются из $U[-0.5; 0.5]$ с фиксированным seed.

5) Метрики (исходная шкала, n — число примеров):

$$\mathrm{MAE} = (1/n) \cdot \Sigma |x - \hat{x}|, \quad \mathrm{RMSE} = \sqrt{((1/n) \cdot \Sigma (x - \hat{x})^2)}$$

$$\mathrm{MAPE} = (100\ \%/n) \cdot \Sigma\ |x - \hat{x}| \ / \ |x|$$

$$\mathrm{sMAPE} = (100\ \%/n) \cdot \Sigma\ 2|x - \hat{x}| \ / \ (|x| + |\hat{x}|)$$

Ряд проходит через ноль (меняет знак), поэтому основная относительная метрика — sMAPE. В MAPE знаменатель $|x|$ при $x \to 0$ стремится к нулю, и отдельные точки дают сколь угодно большие проценты (на нашем ряду есть значения $x(161) = 0.049$, $x(153) = -0.031$). sMAPE нормирует ошибку на среднее $|x|$ и $|\hat{x}|$ и ограничена 200 %. MAPE программа выводит только для сравнения.

6) Выбор скорости обучения. Проверены $\eta \in \{0.01; 0.03; 0.1; 0.3\}$ при $p = 6$, $N = 4$ (по 5 запусков, python ts_lab4.py --extra):

η	Эпох: среднее ± std	Мин-макс	MAE тест	Не достигли Ee
0.01	4160 ± 230	3908-4490	0.0278 ± 0.0004	0 из 5
0.03	1398 ± 78	1315-1513	0.0277 ± 0.0004	0 из 5
0.1	435 ± 26	411-476	0.0275 ± 0.0005	0 из 5
0.3	174 ± 12	162-194	0.0275 ± 0.0008	0 из 5

Все значения дают одинаковую точность и сходятся во всех запусках, а число эпох обратно пропорционально η. Выбрана $\eta = 0.3$ — максимальная из допустимого диапазона: обучение в 24 раза быстрее, чем при $\eta = 0.01$, без потери точности и устойчивости (расходимости и колебаний, мешающих достижению Ee, не наблюдается).

Код программы (ts_lab4.py):

```
r"""
Лабораторная работа 4
Многослойный персептрон p-N-1 для прогнозирования временного ряда:
влияние размера окна p и числа скрытых нейронов N.


Вариант 6: x(k) = a·cos(b·k) + c·sin(d·k),  a = 1.2, b = 0.28, c = 0.6, d = 0.65,  k = 0..199
           p = 6, N = 4
Обучающий участок: k = 0..139 (70 %), тестовый: k = 140..199 (30 %).
Сеть по p предыдущим значениям предсказывает следующее: x(k) = F(x(k-p), ..., x(k-1)).
Скрытый слой - сигмоида, выходной - линейный, функция потерь - MSE, онлайн-обучение
с перемешиванием примеров в каждой эпохе. Остановка: Es <= Ee = 1e-4 (нормализованная шкала) или 20000 эпох.
для запуска -
cd "C:\Users\user\Desktop\4 курс 1\MRZVS\4labPC"
.venv\Scripts\python ts_lab4.py            (графики сохраняются в папку figures, затем открываются окна)
.venv\Scripts\python ts_lab4.py --no-show  (без окон с графиками)
.venv\Scripts\python ts_lab4.py --no-input (без интерактивного режима)
.venv\Scripts\python ts_lab4.py --extra    (+ подбор η и эксперимент с шумом, около 5 минут)
"""


import json
import os
import sys
import numpy as np


if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")


A, B, C, D = 1.2, 0.28, 0.6, 0.65
P_VAR, N_VAR = 6, 4
N_POINTS = 200
K_SPLIT = 140                 # k = 0..139 - обучение, k = 140..199 - тест


LR = 0.3                      # выбрано по результатам подбора (--extra)
INIT = 0.5                    # веса и пороги из U[-INIT; INIT]
EE = 1e-4
MAX_EPOCHS = 20000
SEEDS = [1, 2, 3, 4, 5]
HORIZON = 30


BASE_DIR = os.path.dirname(os.path.abspath(__file__))
FIG_DIR = os.path.join(BASE_DIR, "figures")
RESULTS_DIR = os.path.join(BASE_DIR, "results")
EXTRA_FILE = os.path.join(RESULTS_DIR, "extra_stats.json")




# ---------------------------------------------------------------------------------------
# Данные
# ---------------------------------------------------------------------------------------
def generate_series(n=N_POINTS):
    k = np.arange(n)
    return A * np.cos(B * k) + C * np.sin(D * k)




def fit_minmax(train):
    """Параметры нормализации берутся только по обучающему участку."""
    return float(np.min(train)), float(np.max(train))




def normalize(x, x_min, x_max):
    return (np.asarray(x, dtype=float) - x_min) / (x_max - x_min)




def denormalize(x_norm, x_min, x_max):
    return np.asarray(x_norm, dtype=float) * (x_max - x_min) + x_min




def make_windows(series, p, k_from, k_to):
    """Скользящее окно: вход - p значений x(k-p)..x(k-1) (по времени), цель - x(k), k = k_from..k_to-1."""
    k_from = max(k_from, p)
    X = np.array([series[k - p:k] for k in range(k_from, k_to)])
    y = np.array([series[k] for k in range(k_from, k_to)])
    return X, y, np.arange(k_from, k_to)




# ---------------------------------------------------------------------------------------
# Многослойный персептрон p-N-1 (сразу S независимых сетей - по одной на seed)
# ---------------------------------------------------------------------------------------
def sigmoid(s):
    return 1.0 / (1.0 + np.exp(-s))
```

```
def init_net(p, n_hidden, seeds, init=INIT):
    S = len(seeds)
    net = {"W1": np.zeros((S, p, n_hidden)), "b1": np.zeros((S, n_hidden)),
           "W2": np.zeros((S, n_hidden)), "b2": np.zeros(S)}
    for i, s in enumerate(seeds):
        rng = np.random.default_rng(s)
        net["W1"][i] = rng.uniform(-init, init, (p, n_hidden))
        net["b1"][i] = rng.uniform(-init, init, n_hidden)
        net["W2"][i] = rng.uniform(-init, init, n_hidden)
        net["b2"][i] = rng.uniform(-init, init)
    return net




def take(net, idx):
    return {name: value[idx] for name, value in net.items()}




def forward(net, x):
    """Прямой проход: x (S, p) - по одному примеру на каждую из S сетей.
    h_j = σ(Σ_i w1_ij·x_i + b1_j),  y = Σ_j w2_j·h_j + b2 (линейный выход)."""
    h = sigmoid(np.einsum("si,sij->sj", x, net["W1"]) + net["b1"])
    y = np.einsum("sj,sj->s", h, net["W2"]) + net["b2"]
    return h, y




def forward_all(net, X):
    """Прямой проход всей выборки X (L, p) через каждую из S сетей: выход (S, L)."""
    h = sigmoid(np.einsum("li,sij->slj", X, net["W1"]) + net["b1"][:, None, :])
    return np.einsum("slj,sj->sl", h, net["W2"]) + net["b2"][:, None]




def backward(net, x, h, y, d, lr):
    """Обратный проход для одного примера на сеть (онлайн-режим), E = ½(d − y)²:
       δ_out = d − y                          (линейный выход, f'(s) = 1)
       δ_j   = δ_out · w2_j · h_j(1 − h_j)    (сигмоидный скрытый слой)
       W2 += η·δ_out·h,  b2 += η·δ_out,  W1 += η·x ⊗ δ,  b1 += η·δ"""
    d_out = d - y
    d_hid = d_out[:, None] * net["W2"] * h * (1.0 - h)
    net["W2"] += lr * d_out[:, None] * h
    net["b2"] += lr * d_out
    net["W1"] += lr * x[:, :, None] * d_hid[:, None, :]
    net["b1"] += lr * d_hid




def mse(net, X, y):
    return np.mean((forward_all(net, X) - y) ** 2, axis=1)




def train(X, y, n_hidden, seeds, lr=LR, ee=EE, max_epochs=MAX_EPOCHS, noise=0.0):
    """Онлайн-обучение S сетей (по одной на seed); у каждой сети свой порядок примеров в каждой эпохе.
    Es - средняя квадратичная ошибка по всей обучающей выборке после эпохи; остановка при Es <= Ee.
    noise > 0 - к входам при обучении добавляется гауссов шум (эксперимент --extra)."""
    p = X.shape[1]
    net = init_net(p, n_hidden, seeds)
    rngs = [np.random.default_rng(s) for s in seeds]
    noise_rng = np.random.default_rng(12345)
    S, L = len(seeds), len(y)
    epochs = np.full(S, max_epochs)
    active = np.ones(S, dtype=bool)
    history = [[] for _ in range(S)]
    for epoch in range(1, max_epochs + 1):
        idx = np.flatnonzero(active)
        if len(idx) == 0:
            break
        sub = take(net, idx)
        order = np.array([rngs[i].permutation(L) for i in idx])
        for step in range(L):
            x = X[order[:, step]]
            if noise:
                x = x + noise * noise_rng.standard_normal(x.shape)
            h, out = forward(sub, x)
            backward(sub, x, h, out, y[order[:, step]], lr)
        for name in net:
            net[name][idx] = sub[name]
        es = mse(net, X, y)
        for i in idx:
            history[i].append(float(es[i]))
        done = active & (es <= ee)
        epochs[done] = epoch
        active &= ~done
    return net, epochs, mse(net, X, y), history
```

```
def single(net, i):
    return {name: value[i:i + 1] for name, value in net.items()}



def predict(net, X):
    """Прогноз одной сети (S = 1) для выборки X: (L, p) -> (L,)."""
    return forward_all(net, np.atleast_2d(X))[0]



# ---------------------------------------------------------------------------------------
# Метрики (в исходной шкале)
# ---------------------------------------------------------------------------------------
def mae(y_true, y_pred):
    return float(np.mean(np.abs(y_true - y_pred)))



def rmse(y_true, y_pred):
    return float(np.sqrt(np.mean((y_true - y_pred) ** 2)))



def mape(y_true, y_pred):
    return float(100.0 * np.mean(np.abs((y_true - y_pred) / y_true)))



def smape(y_true, y_pred):
    return float(100.0 * np.mean(2.0 * np.abs(y_true - y_pred) / (np.abs(y_true) + np.abs(y_pred))))



def metrics(y_true, y_pred):
    return {"MAE": mae(y_true, y_pred), "RMSE": rmse(y_true, y_pred), "sMAPE": smape(y_true, y_pred),
            "MAPE": mape(y_true, y_pred)}



# ---------------------------------------------------------------------------------------
# Эксперименты
# ---------------------------------------------------------------------------------------
class Data:
    def __init__(self):
        self.k = np.arange(N_POINTS)
        self.x = generate_series()
        self.x_min, self.x_max = fit_minmax(self.x[:K_SPLIT])
        self.xn = normalize(self.x, self.x_min, self.x_max)


    def windows(self, p):
        Xtr, ytr, ktr = make_windows(self.xn, p, 0, K_SPLIT)
        Xte, yte, kte = make_windows(self.xn, p, K_SPLIT, N_POINTS)
        return Xtr, ytr, ktr, Xte, yte, kte


    def to_real(self, v):
        return denormalize(v, self.x_min, self.x_max)



def run_config(data, p, n_hidden, seeds=SEEDS, lr=LR, noise=0.0):
    Xtr, ytr, ktr, Xte, yte, kte = data.windows(p)
    net, epochs, es, history = train(Xtr, ytr, n_hidden, seeds, lr=lr, noise=noise)
    runs = []
    for i, s in enumerate(seeds):
        one = single(net, i)
        pred_tr = data.to_real(predict(one, Xtr))
        pred_te = data.to_real(predict(one, Xte))
        runs.append({"seed": s, "net": one, "epochs": int(epochs[i]), "Es": float(es[i]),
                     "converged": bool(es[i] <= EE), "history": history[i],
                     "train": metrics(data.x[ktr], pred_tr), "test": metrics(data.x[kte], pred_te),
                     "pred_tr": pred_tr, "pred_te": pred_te, "ktr": ktr, "kte": kte})
    return {"p": p, "N": n_hidden, "lr": lr, "noise": noise, "runs": runs}



def summary(cfg):
    ep = np.array([r["epochs"] for r in cfg["runs"]])
    te = np.array([r["test"]["MAE"] for r in cfg["runs"]])
    tr = np.array([r["train"]["MAE"] for r in cfg["runs"]])
    return {"p": cfg["p"], "N": cfg["N"], "lr": cfg["lr"], "noise": cfg["noise"],
            "ep_mean": float(ep.mean()), "ep_std": float(ep.std()),
            "ep_min": int(ep.min()), "ep_max": int(ep.max()),
            "mae_mean": float(te.mean()), "mae_std": float(te.std()), "train_mae_mean": float(tr.mean()),
            "failed": int(sum(not r["converged"] for r in cfg["runs"])), "n": len(cfg["runs"])}
```

```
def recursive_forecast(net, data, p, k_start=K_SPLIT, horizon=HORIZON):
    """Рекурсивный прогноз: окно начинается с известных значений x(k_start-p..k_start-1),
    каждое новое предсказание добавляется в окно вместо самого старого значения."""
    window = list(data.xn[k_start - p:k_start])
    preds = []
    for _ in range(horizon):
        y = float(predict(net, np.array(window))[0])
        preds.append(y)
        window = window[1:] + [y]
    return data.to_real(np.array(preds))




def print_base(data, run, p, n_hidden):
    print(f"=== Базовый эксперимент: p = {p}, N = {n_hidden}, η = {LR}, seed = {run['seed']} ===")
    print(f"Эпох: {run['epochs']}  |  критерий Es <= {EE:g} достигнут: {'да' if run['converged'] else 'нет'}")
    print(f"Итоговая Es (нормализованная шкала) = {run['Es']:.3e}")
    print(f"{'участок':<10}{'MAE':>9}{'RMSE':>9}{'sMAPE, %':>10}{'MAPE, %':>10}")
    for part in ("train", "test"):
        m = run[part]
        print(f"{'обучение' if part == 'train' else 'тест':<10}{m['MAE']:>9.4f}{m['RMSE']:>9.4f}"
              f"{m['sMAPE']:>10.2f}{m['MAPE']:>10.1f}")
    print()




def print_study(title, rows, key):
    print(f"=== {title} (по {rows[0]['n']} запусков, seed {SEEDS[0]}-{SEEDS[-1]}) ===")
    print(f"{key:>4}{'эпох: среднее ± std':>24}{'мин-макс':>14}{'MAE тест: среднее ± std':>28}"
          f"{'MAE обуч.':>11}{'не достигли Ee':>16}")
    for r in rows:
        print(f"{r[key]:>4}{r['ep_mean']:>15.0f} ± {r['ep_std']:<6.0f}{r['ep_min']:>7}-{r['ep_max']:<6}"
              f"{r['mae_mean']:>19.4f} ± {r['mae_std']:<6.4f}{r['train_mae_mean']:>11.4f}"
              f"{r['failed']:>10} из {r['n']}")
    print()




# ---------------------------------------------------------------------------------------
# Дополнительное исследование (--extra): подбор η и обучение с шумом на входах
# ---------------------------------------------------------------------------------------
EXTRA_LRS = [0.01, 0.03, 0.1, 0.3]
EXTRA_NOISE = [0.0, 0.005, 0.01, 0.02]




def extra_study(data):
    out = {"lr": [], "noise": []}
    for lr in EXTRA_LRS:
        out["lr"].append(summary(run_config(data, P_VAR, N_VAR, lr=lr)))
        print(f"  η = {lr}: готово", flush=True)
    for nz in EXTRA_NOISE:
        out["noise"].append(summary(run_config(data, P_VAR, N_VAR, noise=nz)))
        print(f"  шум σ = {nz}: готово", flush=True)
    os.makedirs(RESULTS_DIR, exist_ok=True)
    with open(EXTRA_FILE, "w", encoding="utf-8") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    return out




def load_extra():
    if os.path.exists(EXTRA_FILE):
        with open(EXTRA_FILE, encoding="utf-8") as f:
            return json.load(f)
    return None




def print_extra(extra):
    print_study(f"Подбор скорости обучения (p = {P_VAR}, N = {N_VAR})", extra["lr"], "lr")
    print("=== Шум на входах при обучении (σ в нормализованной шкале), p = 6, N = 4, η = 0.3 ===")
    print(f"{'σ':>6}{'эпох: среднее ± std':>24}{'MAE тест':>12}{'MAE обуч.':>11}{'не достигли Ee':>16}")
    for r in extra["noise"]:
        print(f"{r['noise']:>6}{r['ep_mean']:>15.0f} ± {r['ep_std']:<6.0f}{r['mae_mean']:>12.4f}"
              f"{r['train_mae_mean']:>11.4f}{r['failed']:>10} из {r['n']}")
    print()




# ---------------------------------------------------------------------------------------
# Режим функционирования
# ---------------------------------------------------------------------------------------
def functioning(net, data, p, values=None, k=None):
    """Прогноз по окну: либо p значений в исходной шкале, либо номер k (окно x(k-p)..x(k-1) из ряда)."""
    if k is not None:
        values = data.x[k - p:k]
    values = np.asarray(values, dtype=float)
    x_norm = normalize(values, data.x_min, data.x_max)
    y_norm = float(predict(net, x_norm)[0])
    y_real = float(data.to_real(y_norm))
    true = float(data.x[k]) if k is not None and k < N_POINTS else None
    return x_norm, y_norm, y_real, true
```

```
def print_functioning(net, data, p, k=None, values=None, label=""):
    x_norm, y_norm, y_real, true = functioning(net, data, p, values=values, k=k)
    head = f"k = {k}" if k is not None else "пользовательское окно"
    print(f"--- {head} {label}".rstrip())
    if k is None:
        print(f"  входы:               {' '.join(f'{v:.3f}' for v in values)}")
    print(f"  нормализованные входы: {' '.join(f'{v:.4f}' for v in x_norm)}")
    print(f"  прогноз (норм.)   = {y_norm:.4f}")
    print(f"  прогноз (исходн.) = {y_real:.4f}")
    if true is not None:
        print(f"  истинное x({k})   = {true:.4f}   |ошибка| = {abs(true - y_real):.4f}")



def interactive_loop(net, data, p):
    print("=== Интерактивный режим ===")
    print(f"Введите номер k ({p}..{N_POINTS - 1}) - окно x(k-{p})..x(k-1) берётся из ряда,")
    print(f"или {p} значений ряда в исходной шкале через пробел (от старого к новому). Выход - 'q'.")
    while True:
        try:
            raw = input("> ").strip()
        except EOFError:
            break
        if not sys.stdin.isatty():
            print(raw)
        if raw.lower() in ("q", "quit", "exit"):
            break
        parts = raw.replace(",", " ").split()
        try:
            nums = [float(v) for v in parts]
        except ValueError:
            print("Не удалось распознать числа.")
            continue
        if len(nums) == 1 and nums[0].is_integer() and p <= nums[0] < N_POINTS:
            print_functioning(net, data, p, k=int(nums[0]))
        elif len(nums) == p:
            print_functioning(net, data, p, values=nums)
        else:
            print(f"Нужно ввести номер k из [{p}; {N_POINTS - 1}] или ровно {p} чисел.")



# ---------------------------------------------------------------------------------------------
# Визуализация
# ---------------------------------------------------------------------------------------------
COLORS = {"true": "black", "train": "#1f77b4", "test": "#d62728"}



def fig_series(data, path):
    import matplotlib.pyplot as plt
    fig, ax = plt.subplots(figsize=(12, 4.5))
    ax.plot(data.k, data.x, color="black", lw=1.4, marker="o", ms=2.5, label="x(k)")
    ax.axvline(K_SPLIT - 0.5, color="gray", ls="--", lw=1.5)
    ax.axvspan(-0.5, K_SPLIT - 0.5, color="#1f77b4", alpha=0.06, label="обучающий участок (k = 0..139)")
    ax.axvspan(K_SPLIT - 0.5, N_POINTS - 0.5, color="#d62728", alpha=0.06, label="тестовый участок (k = 140..199)")
    for v, txt in ((data.x_min, "min(train)"), (data.x_max, "max(train)")):
        ax.axhline(v, color="#1f77b4", ls=":", lw=1)
        ax.text(N_POINTS - 1, v, txt, ha="right", va="bottom", fontsize=8, color="#1f77b4")
    ax.set_xlim(-1, N_POINTS)
    ax.set_xlabel("k")
    ax.set_ylabel("x(k)")
    ax.set_title(f"Ряд x(k) = {A}·cos({B}·k) + {C}·sin({D}·k)")
    ax.grid(True, alpha=0.3)
    ax.legend(loc="upper left", fontsize=9, ncol=3)
    fig.tight_layout()
    fig.savefig(path, dpi=150)



def fig_one_step(data, run, path):
    import matplotlib.pyplot as plt
    fig, ax = plt.subplots(figsize=(12, 4.8))
    ax.plot(data.k, data.x, color="black", lw=2.2, label="эталон x(k)")
    ax.plot(run["ktr"], run["pred_tr"], color=COLORS["train"], lw=1.2, ls="--", marker=".", ms=4,
            label=f"прогноз на обучении (MAE = {run['train']['MAE']:.4f})")
    ax.plot(run["kte"], run["pred_te"], color=COLORS["test"], lw=1.2, ls="--", marker=".", ms=4,
            label=f"прогноз на тесте (MAE = {run['test']['MAE']:.4f})")
    ax.axvline(K_SPLIT - 0.5, color="gray", ls="--", lw=1.5)
    ax.text(K_SPLIT + 1, ax.get_ylim()[1] * 0.92, "train | test", color="gray")
    ax.set_xlabel("k")
    ax.set_ylabel("x(k)")
    ax.set_title(f"Одношаговый прогноз сети {P_VAR}-{N_VAR}-1 на обучающем и тестовом участках")
    ax.grid(True, alpha=0.3)
    ax.legend(loc="lower left", fontsize=9)
    fig.tight_layout()
    fig.savefig(path, dpi=150)



def fig_convergence(curves, path):
    import matplotlib.pyplot as plt
```

```
    fig, ax = plt.subplots(figsize=(10, 5.5))
    for label, run, color in curves:
        ep = np.arange(1, len(run["history"]) + 1)
        ax.plot(ep, run["history"], color=color, lw=2,
                label=f"{label}: {run['epochs']} эпох" + ("" if run["converged"] else " (Ee не достигнута)"))
    ax.axhline(EE, color="black", ls=":", lw=1.3, label=f"Ee = {EE:g}")
    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.set_xlabel("Номер эпохи (лог. шкала)")
    ax.set_ylabel("Es - средняя ошибка (норм. шкала, лог.)")
    ax.set_title(f"Сходимость при разных размерах окна (N = {N_VAR}, η = {LR}, seed = 1)")
    ax.grid(True, which="both", alpha=0.3)
    ax.legend(fontsize=9)
    fig.tight_layout()
    fig.savefig(path, dpi=150)




def fig_scatter(run, data, path):
    import matplotlib.pyplot as plt
    true = data.x[run["kte"]]
    fig, ax = plt.subplots(figsize=(6.2, 6))
    ax.scatter(true, run["pred_te"], color=COLORS["test"], s=28, edgecolors="black", linewidths=0.4,
               label="тестовые примеры")
    lim = [min(true.min(), run["pred_te"].min()) - 0.1, max(true.max(), run["pred_te"].max()) + 0.1]
    ax.plot(lim, lim, color="black", lw=1.2, label="y = x")
    ax.set_xlim(lim)
    ax.set_ylim(lim)
    ax.set_aspect("equal")
    ax.set_xlabel("эталон x(k)")
    ax.set_ylabel("прогноз сети")
    ax.set_title(f"Тест: эталон - прогноз (MAE = {run['test']['MAE']:.4f})")
    ax.grid(True, alpha=0.3)
    ax.legend()
    fig.tight_layout()
    fig.savefig(path, dpi=150)




def fig_recursive(data, rec, one_step, best, threshold, step_bad, path):
    import matplotlib.pyplot as plt
    k = np.arange(K_SPLIT, K_SPLIT + HORIZON)
    true = data.x[k]
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
    ax1.plot(np.arange(K_SPLIT - 20, K_SPLIT + HORIZON), data.x[K_SPLIT - 20:K_SPLIT + HORIZON], color="black",
             lw=2, label="эталон x(k)")
    ax1.plot(k, rec, color="#d62728", lw=1.8, ls="--", marker="o", ms=3.5, label="рекурсивный прогноз")
    ax1.plot(k, one_step, color="#1f77b4", lw=1, ls=":", marker=".", ms=3, label="одношаговый прогноз")
    ax1.axvline(K_SPLIT - 0.5, color="gray", ls="--")
    ax1.set_xlabel("k")
    ax1.set_ylabel("x(k)")
    ax1.set_title(f"Прогноз на {HORIZON} шагов от k = {K_SPLIT} (p = {best['p']}, N = {best['N']})")
    ax1.grid(True, alpha=0.3)
    ax1.legend(fontsize=9)
    steps = np.arange(1, HORIZON + 1)
    ax2.plot(steps, np.abs(true - rec), color="#d62728", marker="o", ms=3.5, label="рекурсивный")
    ax2.plot(steps, np.abs(true - one_step), color="#1f77b4", marker=".", ms=3, label="одношаговый")
    ax2.axhline(threshold, color="black", ls="--", lw=1.2, label=f"5 % размаха ряда = {threshold:.3f}")
    if step_bad is not None:
        ax2.axvline(step_bad, color="gray", ls=":", label=f"превышение на шаге {step_bad}")
    ax2.set_yscale("log")
    ax2.set_xlabel("номер шага прогноза")
    ax2.set_ylabel("|ошибка| (лог. шкала)")
    ax2.set_title("Абсолютная ошибка по шагам")
    ax2.grid(True, which="both", alpha=0.3)
    ax2.legend(fontsize=9)
    fig.tight_layout()
    fig.savefig(path, dpi=150)




def fig_studies(p_rows, n_rows, path):
    import matplotlib.pyplot as plt
    fig, axes = plt.subplots(1, 2, figsize=(13, 5.2), layout="constrained")
    for ax, rows, key, title in ((axes[0], p_rows, "p", f"Размер окна p (N = {N_VAR})"),
                                 (axes[1], n_rows, "N", f"Число скрытых нейронов N (p = {P_VAR})")):
        x = np.arange(len(rows))
        means = [r["mae_mean"] for r in rows]
        bars = ax.bar(x, means, 0.55, yerr=[r["mae_std"] for r in rows], capsize=5, color="#d62728", alpha=0.8,
                      label="MAE на тесте (среднее ± std)")
        for b, r in zip(bars, rows):
            txt = f"{r['mae_mean']:.4f}" + (f"\nне сошлись:\n{r['failed']} из {r['n']}" if r["failed"] else "")
            ax.text(b.get_x() + b.get_width() / 2, b.get_height() / 2, txt, ha="center", va="center", fontsize=8,
                    color="white", fontweight="bold")
        ax.set_ylim(0, max(means) * 1.15)
        ax.set_xticks(x)
        ax.set_xticklabels([f"{key} = {r[key]}" for r in rows])
        ax.set_ylabel("MAE на тесте (исходная шкала)")
        ax2 = ax.twinx()
        ax2.plot(x, [r["ep_mean"] for r in rows], color="#1f77b4", marker="o", lw=2, label="эпох до остановки
(среднее)")
        if key == "p":
            ax2.set_yscale("log")
            ax2.set_ylabel("Эпох (лог. шкала)", color="#1f77b4")
        else:
            ax2.set_ylim(0, max(r["ep_mean"] for r in rows) * 1.3)
            ax2.set_ylabel("Эпох", color="#1f77b4")
        ax.set_title(title)
        h1, l1 = ax.get_legend_handles_labels()
        h2, l2 = ax2.get_legend_handles_labels()
```

```
        ax.legend(h1 + h2, l1 + l2, fontsize=8, loc="upper center", bbox_to_anchor=(0.5, -0.1), ncol=2)
    fig.savefig(path, dpi=150)



# ---------------------------------------------------------------------------------------
if __name__ == "__main__":
    show = "--no-show" not in sys.argv
    if not show:
        import matplotlib
        matplotlib.use("Agg")
    os.makedirs(FIG_DIR, exist_ok=True)


    data = Data()
    print(f"Ряд: x(k) = {A}·cos({B}·k) + {C}·sin({D}·k), k = 0..{N_POINTS - 1};  p = {P_VAR}, N = {N_VAR}")
    print(f"Нормализация по обучающему участку: x_min = {data.x_min:.4f}, x_max = {data.x_max:.4f}")
    print(f"Весь ряд: min = {data.x.min():.4f}, max = {data.x.max():.4f};  тест в норм. шкале: "
          f"[{data.xn[K_SPLIT:].min():.4f}; {data.xn[K_SPLIT:].max():.4f}]")
    Xtr, ytr, _, Xte, yte, _ = data.windows(P_VAR)
    print(f"Скользящее окно p = {P_VAR}: обучающих примеров {len(ytr)} (k = {P_VAR}..{K_SPLIT - 1}), "
          f"тестовых {len(yte)} (k = {K_SPLIT}..{N_POINTS - 1})")
    print(f"η = {LR}, веса из U[-{INIT}; {INIT}], Ee = {EE:g}, максимум {MAX_EPOCHS} эпох")
    print()


    p_values = [2, 4, P_VAR, 2 * P_VAR]
    n_values = [1, 2, N_VAR, 2 * N_VAR]
    configs = {}
    for p in p_values:
        configs[(p, N_VAR)] = run_config(data, p, N_VAR)
    for n in n_values:
        if (P_VAR, n) not in configs:
            configs[(P_VAR, n)] = run_config(data, P_VAR, n)


    base = configs[(P_VAR, N_VAR)]["runs"][0]
    print_base(data, base, P_VAR, N_VAR)


    p_rows = [summary(configs[(p, N_VAR)]) for p in p_values]
    n_rows = [summary(configs[(P_VAR, n)]) for n in n_values]
    print_study(f"Исследование окна p (N = {N_VAR})", p_rows, "p")
    print_study(f"Исследование скрытого слоя N (p = {P_VAR})", n_rows, "N")


    best = min((summary(c) for c in configs.values()), key=lambda r: r["mae_mean"])
    best_cfg = configs[(best["p"], best["N"])]
    best_run = sorted(best_cfg["runs"], key=lambda r: r["test"]["MAE"])[len(best_cfg["runs"]) // 2]
    rec = recursive_forecast(best_run["net"], data, best["p"])
    true = data.x[K_SPLIT:K_SPLIT + HORIZON]
    one_step = best_run["pred_te"][:HORIZON]
    err = np.abs(true - rec)
    span = float(data.x.max() - data.x.min())
    threshold = 0.05 * span
    bad = np.flatnonzero(err > threshold)
    step_bad = int(bad[0]) + 1 if len(bad) else None
    print(f"=== Рекурсивный прогноз на {HORIZON} шагов от k = {K_SPLIT}: лучшая конфигурация p = {best['p']}, "
          f"N = {best['N']} (seed {best_run['seed']}) ===")
    print(f"{'шаг':>4}{'k':>5}{'эталон':>10}{'рекурсивный':>13}{'|ошибка|':>10}{'одношаговый':>13}{'|ошибка|':>10}")
    for s in range(HORIZON):
        print(f"{s + 1:>4}{K_SPLIT + s:>5}{true[s]:>10.4f}{rec[s]:>13.4f}{err[s]:>10.4f}"
              f"{one_step[s]:>13.4f}{abs(true[s] - one_step[s]):>10.4f}")
    print(f"Порог 5 % размаха ряда: 0.05·{span:.4f} = {threshold:.4f};  "
          + (f"превышен впервые на шаге {step_bad}" if step_bad else f"не превышен за {HORIZON} шагов"))
    print(f"MAE рекурсивного прогноза = {err.mean():.4f}, одношагового на тех же k = "
          f"{np.mean(np.abs(true - one_step)):.4f};  макс. |ошибка| рекурсивного = {err.max():.4f}")
    full = N_POINTS - K_SPLIT
    err_full = np.abs(data.x[K_SPLIT:] - recursive_forecast(best_run["net"], data, best["p"], horizon=full))
    bad_full = np.flatnonzero(err_full > threshold)
    print(f"Продолжение рекурсии до конца теста ({full} шагов): макс. |ошибка| = {err_full.max():.4f}, "
          f"MAE = {err_full.mean():.4f}; " + (f"порог 5 % впервые превышен на шаге {bad_full[0] + 1}"
                                              if len(bad_full) else "порог 5 % не превышен"))
    print()


    extra = extra_study(data) if "--extra" in sys.argv else load_extra()
    if extra is not None:
        print_extra(extra)


    paths = {"series": "fig1_series.png", "one_step": "fig2_one_step.png", "convergence": "fig3_convergence.png",
             "scatter": "fig4_scatter.png", "recursive": "fig5_recursive.png", "studies": "fig6_studies.png"}
    paths = {k: os.path.join(FIG_DIR, v) for k, v in paths.items()}
    fig_series(data, paths["series"])
    fig_one_step(data, base, paths["one_step"])
    fig_convergence([(f"p = {P_VAR} (базовая)", base, "#1f77b4"),
                     ("p = 2", configs[(2, N_VAR)]["runs"][0], "#d62728"),
                     (f"p = {2 * P_VAR}", configs[(2 * P_VAR, N_VAR)]["runs"][0], "#2ca02c")],
                    paths["convergence"])
    fig_scatter(base, data, paths["scatter"])
    fig_recursive(data, rec, one_step, best, threshold, step_bad, paths["recursive"])
    fig_studies(p_rows, n_rows, paths["studies"])
    print("Графики сохранены:")
    for v in paths.values():
        print("  ", v)
    print()


    print(f"=== Режим функционирования (базовая сеть p = {P_VAR}, N = {N_VAR}) ===")
    for k in (20, 70, 120):
```

```
    print_functioning(base["net"], data, P_VAR, k=k, label="(обучающий участок)")
for k in (150, 175, 199):
    print_functioning(base["net"], data, P_VAR, k=k, label="(тестовый участок)")
custom = [round(float(v), 3) for v in generate_series(260)[250:256]]
print_functioning(base["net"], data, P_VAR, values=custom, label="(значения формулы при k = 250..255; x(256) = "
                  f"{generate_series(257)[256]:.4f})")
outlier = list(data.x[164:170].round(3))
outlier[3] = 3.0
print_functioning(base["net"], data, P_VAR, values=outlier, label="(окно k = 170 с выбросом 3.0 на 4-й позиции; "
                  f"истинное x(170) = {data.x[170]:.4f})")
print()


if "--no-input" not in sys.argv:
    interactive_loop(base["net"], data, P_VAR)


if show:
    import matplotlib.pyplot as plt
    plt.show()
```

Результат:

Запуск программы: параметры нормализации и окна, базовый эксперимент (p = 6, N = 4, η = 0.3, seed = 1):

```
(.venv) PS C:\Users\user\Desktop\4 курс 1\MRZVS\4labPC> .venv\Scripts\python ts_lab4.py
Ряд: x(k) = 1.2·cos(0.28·k) + 0.6·sin(0.65·k), k = 0..199;  p = 6, N = 4
Нормализация по обучающему участку: x_min = -1.7840, x_max = 1.7830
Весь ряд: min = -1.7958, max = 1.7989;  тест в норм. шкале: [-0.0033; 1.0045]
Скользящее окно p = 6: обучающих примеров 134 (k = 6..139), тестовых 60 (k = 140..199)
η = 0.3, веса из U[-0.5; 0.5], Ee = 0.0001, максимум 20000 эпох

=== Базовый эксперимент: p = 6, N = 4, η = 0.3, seed = 1 ===
Эпох: 194  |  критерий Es <= 0.0001 достигнут: да
Итоговая Es (нормализованная шкала) = 9.219e-05
участок         MAE     RMSE  sMAPE, %   MAPE, %
обучение     0.0286   0.0342      7.63       7.8
тест         0.0276   0.0338      9.99      10.1
```

Таблица 1 - Результаты базового эксперимента (метрики в исходной шкале)

Участок	MAE	RMSE	sMAPE, %	MAPE, % (справочно)
Обучающий (134 примера)	0.0286	0.0342	7.63	7.8
Тестовый (60 примеров)	0.0276	0.0338	9.99	10.1

Обучение завершилось на 194-й эпохе по критерию $Es = 9.22 \cdot 10^{-5} \leq 10^{-4}$. Ошибки на обучающем и тестовом участках практически совпадают.

Исследования размера окна и числа скрытых нейронов (по 5 запусков, seed 1-5):

```
=== Исследование окна p (N = 4) (по 5 запусков, seed 1-5) ===
   p     эпох: среднее ± std      мин-макс     MAE тест: среднее ± std  MAE обуч.  не достигли Ee
   2          20000 ± 0       20000-20000              0.0968 ± 0.0027     0.0985         5 из 5
   4           2542 ± 203      2165-2713               0.0262 ± 0.0005     0.0282         0 из 5
   6            174 ± 12        162-194                0.0275 ± 0.0008     0.0284         0 из 5
  12             10 ± 0           9-10                 0.0233 ± 0.0042     0.0231         0 из 5

=== Исследование скрытого слоя N (p = 6) (по 5 запусков, seed 1-5) ===
   N     эпох: среднее ± std      мин-макс     MAE тест: среднее ± std  MAE обуч.  не достигли Ee
   1           186 ± 4         182-193                0.0273 ± 0.0009     0.0284         0 из 5
   2           190 ± 16        168-207                0.0274 ± 0.0007     0.0289         0 из 5
   4           174 ± 12        162-194                0.0275 ± 0.0008     0.0284         0 из 5
   8           180 ± 7         167-189                0.0255 ± 0.0014     0.0269         0 из 5
```

Таблица 2 - Исследование размера окна p (N = 4, η = 0.3)

p	Эпох: среднее ± std	Мин-макс	MAE тест: среднее ± std	MAE обуч.	Не достигли Ee
2	20000 ± 0	20000	0.0968 ± 0.0027	0.0985	5 из 5
4	2542 ± 203	2165-2713	0.0262 ± 0.0005	0.0282	0 из 5
6	174 ± 12	162-194	0.0275 ± 0.0008	0.0284	0 из 5
12	10 ± 0	9-10	0.0233 ± 0.0042	0.0231	0 из 5

Таблица 3 - Исследование числа скрытых нейронов N (p = 6, η = 0.3)

N	Эпох: среднее ± std	Мин-макс	MAE тест: среднее ± std	MAE обуч.	Не достигли Ee
1	186 ± 4	182-193	0.0273 ± 0.0009	0.0284	0 из 5
2	190 ± 16	168-207	0.0274 ± 0.0007	0.0289	0 из 5
4	174 ± 12	162-194	0.0275 ± 0.0008	0.0284	0 из 5
8	180 ± 7	167-189	0.0255 ± 0.0014	0.0269	0 из 5

Ключевое наблюдение — провал при p = 2 и одинаковое качество при всех p ≥ 4 и при любом N от 1 до 8. Это объясняется структурой ряда: сумма двух гармоник точно подчиняется линейному рекуррентному соотношению 4-го порядка $x(k) = 3.514 \cdot x(k-1) - 5.060 \cdot x(k-2) + 3.514 \cdot x(k-3) - x(k-4)$ (проверено программно, невязка $5 \cdot 10^{-14}$). Поэтому четырёх предыдущих значений достаточно для точного прогноза, двух — нет, а зависимость линейна, и с ней справляется даже один нейрон (подробно — в ответах на вопросы 1 и 2). Остаточный MAE ≈ 0.026 задаётся порогом Ee.

Рекурсивный прогноз для лучшей по тестовому MAE конфигурации (p = 12, N = 4; запуск с медианным MAE — seed 2):

```
=== Рекурсивный прогноз на 30 шагов от k = 140: лучшая конфигурация p = 12, N = 4 (seed 2) ===
 шаг    k    эталон  рекурсивный  |ошибка|  одношаговый  |ошибка|
   1  140    0.1474       0.1464    0.0010       0.1464    0.0010
   2  141   -0.5607      -0.5796    0.0189      -0.5791    0.0184
   3  142   -1.1227      -1.1222    0.0005      -1.1147    0.0080
   4  143   -1.4132      -1.3717    0.0416      -1.3665    0.0467
   5  144   -1.4030      -1.3458    0.0572      -1.3608    0.0422
   6  145   -1.1641      -1.1225    0.0416      -1.1580    0.0061
   7  146   -0.8349      -0.8063    0.0286      -0.8469    0.0120
   8  147   -0.5609      -0.5361    0.0247      -0.5620    0.0011
   9  148   -0.4340      -0.4185    0.0155      -0.4235    0.0105
  10  149   -0.4571      -0.4709    0.0139      -0.4541    0.0030
  11  150   -0.5462      -0.5959    0.0497      -0.5654    0.0192
  12  151   -0.5710      -0.6451    0.0741      -0.6080    0.0370
  13  152   -0.4148      -0.5007    0.0860      -0.4549    0.0401
  14  153   -0.0308      -0.1116    0.0807      -0.0596    0.0287
  15  154    0.5302       0.4702    0.0600       0.5306    0.0004
  16  155    1.1326       1.1104    0.0222       1.1620    0.0293
  17  156    1.6038       1.6087    0.0049       1.6269    0.0231
  18  157    1.7989       1.8130    0.0141       1.7986    0.0003
  19  158    1.6562       1.6864    0.0302       1.6627    0.0065
  20  159    1.2209       1.2757    0.0548       1.2528    0.0320
  21  160    0.6277       0.6821    0.0544       0.6541    0.0264
  22  161    0.0493       0.0835    0.0341       0.0456    0.0037
  23  162   -0.3688      -0.3487    0.0201      -0.3887    0.0199
  24  163   -0.5605      -0.5503    0.0102      -0.5876    0.0272
  25  164   -0.5580      -0.5649    0.0069      -0.5959    0.0379
  26  165   -0.4700      -0.4931    0.0231      -0.5076    0.0376
  27  166   -0.4289      -0.4470    0.0181      -0.4441    0.0152
  28  167   -0.5296      -0.5211    0.0084      -0.5129    0.0167
  29  168   -0.7843      -0.7422    0.0421      -0.7458    0.0385
  30  169   -1.1137      -1.0443    0.0694      -1.0679    0.0458
Порог 5 % размаха ряда: 0.05·3.5947 = 0.1797;  не превышен за 30 шагов
MAE рекурсивного прогноза = 0.0336, одношагового на тех же k = 0.0212;  макс. |ошибка| рекурсивного = 0.0860
Продолжение рекурсии до конца теста (60 шагов): макс. |ошибка| = 0.2403, MAE = 0.0666; порог 5 % впервые превышен на шаге 51
```

Дополнительное исследование (--extra): подбор η (см. описание метода) и обучение с гауссовым шумом σ на входах (нормализованная шкала):

```
=== Подбор скорости обучения (p = 6, N = 4) (по 5 запусков, seed 1-5) ===
  lr     эпох: среднее ± std      мин-макс     MAE тест: среднее ± std  MAE обуч.  не достигли Ee
0.01          4160 ± 230      3908-4490              0.0278 ± 0.0004     0.0292         0 из 5
0.03          1398 ± 78       1315-1513              0.0277 ± 0.0004     0.0291         0 из 5
 0.1           435 ± 26        411-476               0.0275 ± 0.0005     0.0288         0 из 5
 0.3           174 ± 12        162-194               0.0275 ± 0.0008     0.0284         0 из 5

=== Шум на входах при обучении (σ в нормализованной шкале), p = 6, N = 4, η = 0.3 ===
     σ     эпох: среднее ± std    MAE тест  MAE обуч.  не достигли Ee
   0.0           174 ± 12          0.0275     0.0284         0 из 5
 0.005           191 ± 10          0.0282     0.0292         0 из 5
  0.01          3180 ± 799         0.0277     0.0292         0 из 5
  0.02         20000 ± 0           0.0764     0.0771         5 из 5
```

Таблица 4 - Обучение с шумом на входах (p = 6, N = 4, η = 0.3, 5 запусков)

σ	Эпох: среднее ± std	MAE тест	MAE обуч.	Не достигли Ee
0	174 ± 12	0.0275	0.0284	0 из 5
0.005	191 ± 10	0.0282	0.0292	0 из 5
0.01	3180 ± 799	0.0277	0.0292	0 из 5
0.02	20000 ± 0	0.0764	0.0771	5 из 5

Блок визуализации.

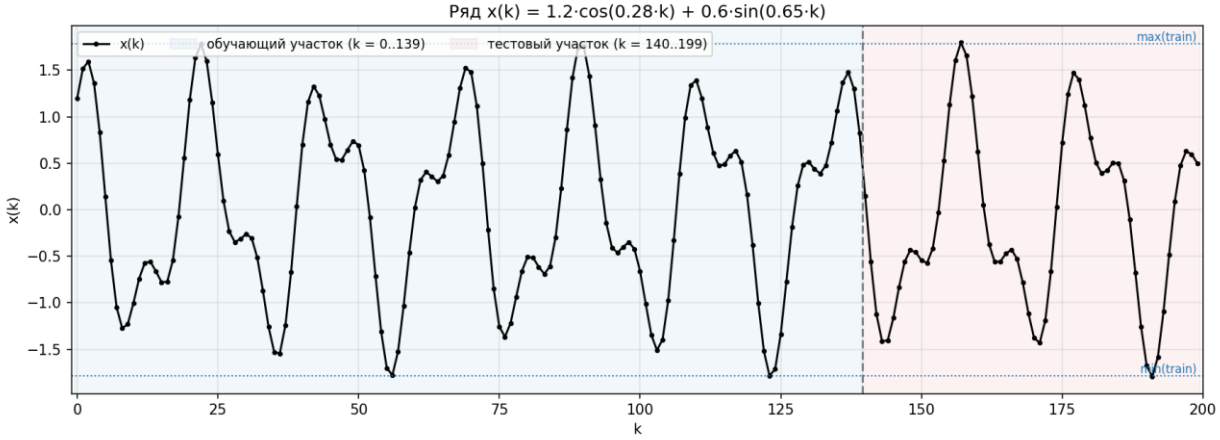


Рисунок 1 - Ряд x(k) = 1.2·cos(0.28k) + 0.6·sin(0.65k) с границей обучающего и тестового участков; пунктир — min и max обучающего участка (параметры нормализации)



Рисунок 2 - Ряд и одношаговый прогноз базовой сети 6-4-1 на обучающем и тестовом участках

Прогноз практически совпадает с эталоном на обоих участках, заметные отклонения есть только в отдельных локальных экстремумах (k ≈ 12, 35, 47, 80, 148), где ряд резко меняет направление.

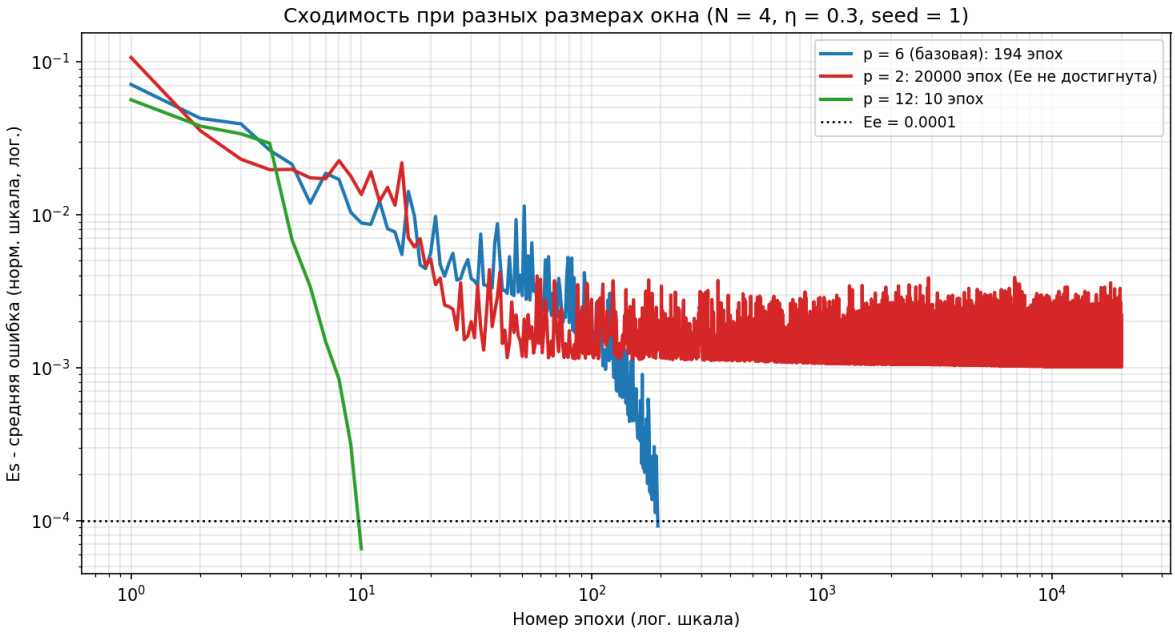


Рисунок 3 - Сходимость: средняя ошибка Es от номера эпохи для базовой конфигурации (p = 6) и для p = 2 и p = 12 (N = 4, seed 1), логарифмические шкалы

При p = 12 ошибка падает ниже Ee за 10 эпох, при p = 6 — за 194, а при p = 2 выходит на «пол» ≈ 10^{-3} и колеблется на нём до лимита 20 000 эпох. Колебания — следствие онлайн-режима с большим η: каждый пример сдвигает веса в свою сторону, а точного решения, которое удовлетворило бы все примеры, при p = 2 нет.

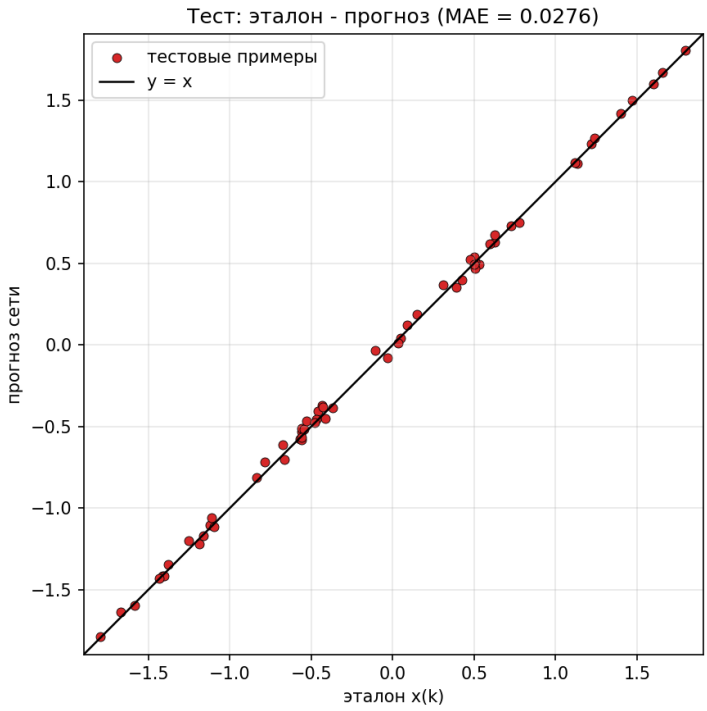


Рисунок 4 - Диаграмма рассеяния «эталон - прогноз» на тестовом участке (базовая сеть) с диагональю y = x

Все точки лежат вплотную к диагонали во всём диапазоне значений, включая крайние (−1.80 и 1.80), выходящие за пределы обучающего диапазона; систематического смещения нет.

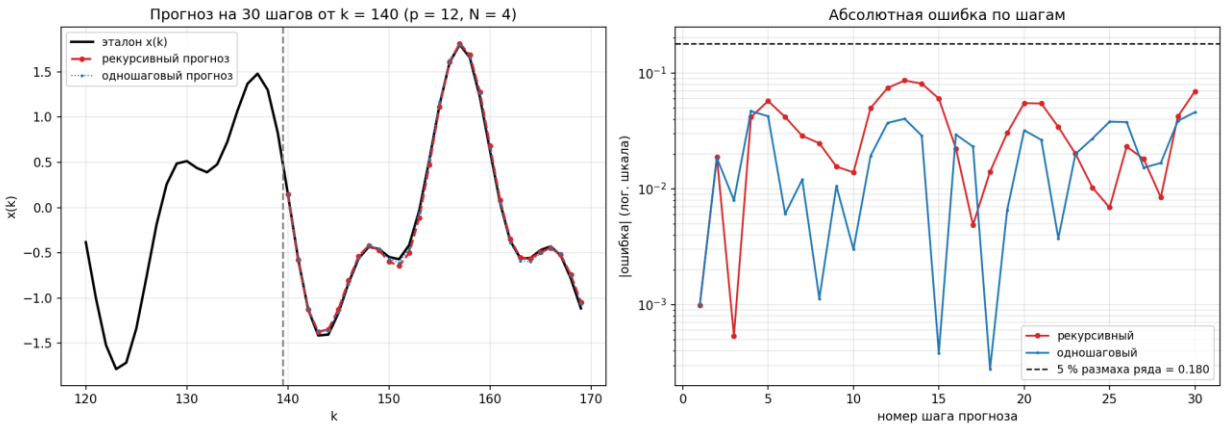


Рисунок 5 - Рекурсивный прогноз на 30 шагов от k = 140 (p = 12, N = 4) в сравнении с эталоном и одношаговым прогнозом (слева) и абсолютная ошибка по шагам с порогом 5 % размаха ряда (справа)

Рекурсивный прогноз визуально совпадает с эталоном на всех 30 шагах. Его ошибка в среднем в 1.6 раза больше одношаговой (MAE 0.0336 против 0.0212), максимум 0.086 на 13-м шаге, порог 0.180 не превышен. При продолжении до 60 шагов порог впервые превышен на 51-м шаге.

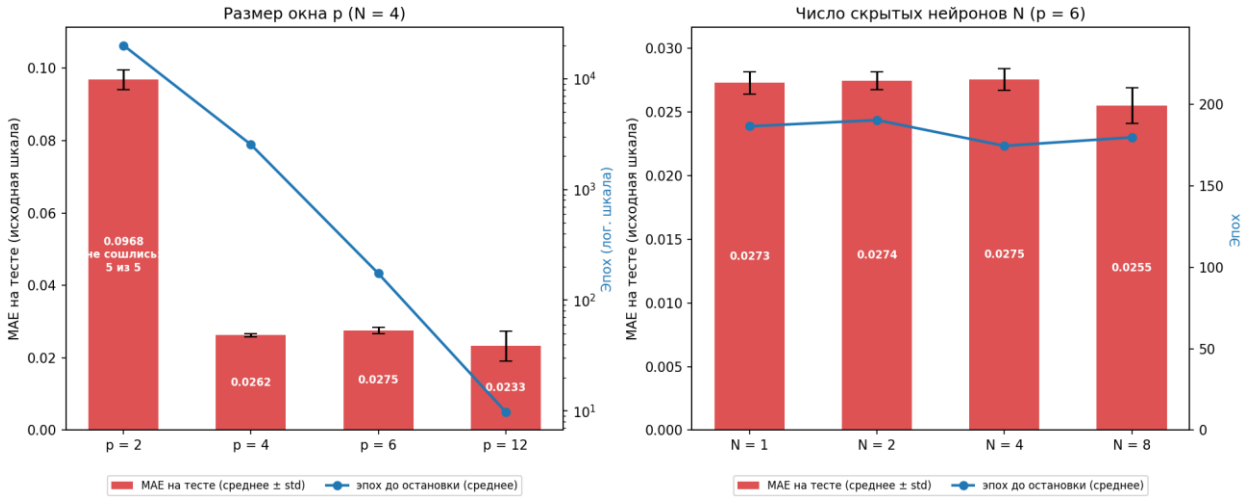


Рисунок 6 - Тестовый MAE (столбцы, среднее ± std) и среднее число эпох (линия) в исследованиях окна p и числа скрытых нейронов N

Режим функционирования базовой сети: 3 окна из обучающего участка, 3 из тестового и 2 пользовательских — значения формулы вне сгенерированного диапазона (k = 250…255) и окно k = 170 с выбросом 3.0. Ниже — интерактивный ввод номера k и произвольного окна:

```
=== Режим функционирования (базовая сеть p = 6, N = 4) ===
--- k = 20 (обучающий участок)
  нормализованные входы: 0.3143 0.2815 0.2834 0.3482 0.4802 0.6561
  прогноз (норм.)   = 0.8288
  прогноз (исходн.) = 1.1723
  истинное x(20)   = 1.1828   |ошибка| = 0.0104
--- k = 70 (обучающий участок)
  нормализованные входы: 0.5856 0.6020 0.6653 0.7647 0.8668 0.9283
  прогноз (норм.)   = 0.9214
  прогноз (исходн.) = 1.5025
  истинное x(70)   = 1.4768   |ошибка| = 0.0256
--- k = 120 (обучающий участок)
  нормализованные входы: 0.6327 0.6368 0.6637 0.6778 0.6444 0.5462
  прогноз (норм.)   = 0.4116
  прогноз (исходн.) = -0.3158
  истинное x(120)   = -0.3823   |ошибка| = 0.0664
--- k = 150 (тестовый участок)
  нормализованные входы: 0.1068 0.1738 0.2661 0.3429 0.3785 0.3720
  прогноз (норм.)   = 0.3549
  прогноз (исходн.) = -0.5183
  истинное x(150)   = -0.5462   |ошибка| = 0.0280
--- k = 175 (тестовый участок)
  нормализованные входы: 0.1879 0.1140 0.0990 0.1668 0.3139 0.5091
  прогноз (норм.)   = 0.7052
  прогноз (исходн.) = 0.7314
  истинное x(175)   = 0.7251   |ошибка| = 0.0063
--- k = 199 (тестовый участок)
  нормализованные входы: 0.1932 0.3655 0.5249 0.6332 0.6764 0.6678
  прогноз (норм.)   = 0.6382
  прогноз (исходн.) = 0.4925
  истинное x(199)   = 0.5001   |ошибка| = 0.0076
--- пользовательское окно (значения формулы при k = 250..255; x(256) = -0.9434)
  входы:               0.304 0.347 0.405 0.347 0.087 -0.375
  нормализованные входы: 0.5854 0.5974 0.6137 0.5974 0.5245 0.3950
  прогноз (норм.)   = 0.2527
  прогноз (исходн.) = -0.8826
--- пользовательское окно (окно k = 170 с выбросом 3.0 на 4-й позиции; истинное x(170) = -1.3772)
  входы:               -0.558 -0.470 -0.429 3.000 -0.784 -1.114
  нормализованные входы: 0.3437 0.3684 0.3799 1.3412 0.2804 0.1878
  прогноз (норм.)   = -0.5381
  прогноз (исходн.) = -3.7034
```

```
=== Интерактивный режим ===
Введите номер k (6..199) - окно x(k-6)..x(k-1) берётся из ряда,
или 6 значений ряда в исходной шкале через пробел (от старого к новому). Выход - 'q'.
> 150
--- k = 150
  нормализованные входы: 0.1068 0.1738 0.2661 0.3429 0.3785 0.3720
  прогноз (норм.)   = 0.3549
  прогноз (исходн.) = -0.5183
  истинное x(150)   = -0.5462   |ошибка| = 0.0280
> 0.5 0.8 1.0 1.2 1.3 1.2
--- пользовательское окно
  входы:               0.500 0.800 1.000 1.200 1.300 1.200
  нормализованные входы: 0.6403 0.7244 0.7805 0.8366 0.8646 0.8366
  прогноз (норм.)   = 0.7813
  прогноз (исходн.) = 1.0027
> q
```

Таблица 5 - Режим функционирования (базовая сеть p = 6, N = 4)

Окно	**Участок**	**Прогноз (норм.)**	**Прогноз $\hat{x}$**	**Истинное x**	**|Ошибка|**
k = 20	обучение	0.8288	1.1723	1.1828	0.0104
k = 70	обучение	0.9214	1.5025	1.4768	0.0256
k = 120	обучение	0.4116	−0.3158	−0.3823	0.0664
k = 150	тест	0.3549	−0.5183	−0.5462	0.0280
k = 175	тест	0.7052	0.7314	0.7251	0.0063
k = 199	тест	0.6382	0.4925	0.5001	0.0076
формула, k = 250…255	вне ряда	0.2527	−0.8826	−0.9434	0.0608
k = 170 с выбросом 3.0	тест	−0.5381	−3.7034	−1.3772	2.3262

На окнах из ряда ошибка составляет 0.006-0.066, т.е. порядка MAE. Окно из значений формулы при k = 250…255, которых сеть не видела, тоже прогнозируется верно (ошибка 0.061): сеть выучила закон ряда, а не конкретные точки. Выброс 3.0 в окне даёт нормализованный вход 1.34 — далеко за пределами [0; 1]. Сеть экстраполирует линейно и выдаёт −3.70 при истинном −1.38, хотя в нормализованной шкале её прогноз (−0.54) тоже выходит за диапазон. Модель не устойчива к выбросам во входных данных: их нужно обнаруживать и устранять до прогноза.

Рукописная часть

1) Графическая схема пайплайна. Из блоков приложения 1 составлена цепочка (образец оформления приведён на рисунке 7, рукописный вариант — на фото ниже):



Рисунок 7 - Схема обработки данных (образец для рукописного варианта)

Обоснование выбора. Ряд генерируется, затем сразу делится на обучающую и тестовую части, и только после этого нормализуется по обучающей части (Min-Max train): так тестовые данные не влияют на модель. Окна строятся из нормализованного ряда. Сеть p-N-1 обучается онлайн-backprop с перемешиванием перед каждой эпохой, критерий Es ≤ 10^{-4} проверяется после эпохи (иначе — следующая эпоха). Обученная сеть даёт рекурсивный прогноз, он денормализуется, и в исходной шкале считаются MAE, RMSE и sMAPE. Не используются: Batch update — по заданию нужен онлайн-режим; Sigmoid output — для регрессии нужен линейный выход; Normalize using whole dataset — утечка информации из теста; MAPE — ряд проходит через ноль.

2) Ручной расчёт одного шага обучения сети 3-2-1 (V = 6, скрытый слой — сигмоида, выход — линейный, $E = \frac{1}{2}(d - y)^2$, $\eta = 0.3$). Исходные данные: X = [0.2; 0.5; 0.8], d = 0.6; W11 = 0.3, W12 = −0.2, W21 = 0.3, W22 = 0.3, W31 = −0.1, W32 = 0.4, b1 = 0.1, b2 = −0.1; W_out1 = 0.5, W_out2 = −1.2, b_out = 0.2.

Таблица 6 - Контрольные значения ручного расчёта (проверка программой)

Величина	**Значение**
S1 = 0.2·0.3 + 0.5·0.3 + 0.8·(−0.1) + 0.1	0.2300
S2 = 0.2·(−0.2) + 0.5·0.3 + 0.8·0.4 − 0.1	0.3300
h1 = σ(S1), h2 = σ(S2)	0.55725, 0.58176
y = 0.5·h1 − 1.2·h2 + 0.2	−0.21949
e = d − y, E = ½e²	0.81949, 0.33578
δ_out = d − y	0.81949
δ1 = δ_out·0.5·h1(1 − h1)	0.81949·0.5·0.24672 = 0.10109
δ2 = δ_out·(−1.2)·h2(1 − h2)	0.81949·(−1.2)·0.24332 = −0.23927
W_out1, W_out2, b_out (новые)	0.63700, −1.05698, 0.44585
W11, W21, W31 (новые)	0.30607, 0.31516, −0.07574
W12, W22, W32 (новые)	−0.21436, 0.26411, 0.34257
b1, b2 (новые)	0.13033, −0.17178
y и E после шага (проверка)	0.23105, 0.06806

После одного шага выход приблизился к цели (−0.219 → 0.231 при d = 0.6), ошибка уменьшилась в 4.9 раза (0.336 → 0.068).

Вопросы для вывода:

1. Как размер окна p влияет на качество прогноза? При каком p ошибка перестаёт уменьшаться? Почему слишком маленькое окно (p = 2) не позволяет точно описать ряд из двух гармоник, а слишком большое не даёт выигрыша?
2. Как число скрытых нейронов N влияет на точность и скорость обучения? Наблюдается ли переобучение (малая ошибка на обучении при заметно большей на тесте)? Достаточно ли минимального N, чтобы воспроизвести сумму двух гармоник?
3. Почему на выходе для задачи прогнозирования используется линейная функция, а не сигмоида (как в ЛР №3)? Что произошло бы при сигмоиде на выходе и при обучении на ненормализованных значениях?
4. Почему нормализация должна опираться только на обучающий участок? К чему приводит выход тестовых значений за диапазон [0; 1] и как это проявляется на ваших данных?
5. Насколько устойчиво обучение к случайной инициализации весов (по сериям из 5 запусков)? Были ли запуски, не достигшие Ee за 20 000 эпох, и от чего это зависит (η, N, p)?
6. Как растёт ошибка рекурсивного многошагового прогноза с увеличением горизонта? Объясните накопление ошибки и сравните одношаговый прогноз с многошаговым. До какого горизонта прогноз можно считать пригодным?
7. Как согласуется различие между ошибкой на обучающем и тестовом участках с понятием обобщающей способности? Какие меры (ранняя остановка, уменьшение N, добавление шума, увеличение обучающей выборки) могли бы улучшить обобщение? Что показал эксперимент с шумом?

1. Как размер окна p влияет на качество прогноза? При каком p ошибка перестаёт уменьшаться? Почему слишком маленькое окно (p = 2) не позволяет точно описать ряд из двух гармоник, а слишком большое не даёт выигрыша?

При p = 2 сеть не достигла Ee ни в одном из 5 запусков: ошибка за 20000 эпох колеблется около $Es \approx 1 \cdot 10^{-3}$, тестовый MAE = 0.0968 ± 0.0027. Уже при p = 4 все запуски сходятся, и тестовый MAE падает в 3.7 раза — до 0.0262 ± 0.0005. При p = 6 и p = 12 ошибка дальше практически не уменьшается: 0.0275 ± 0.0008 и 0.0233 ± 0.0042. Остаточный MAE ≈ 0.026 определяется не окном, а порогом остановки $Ee = 10^{-4}$: обучение прекращается, как только RMSE в

нормализованной шкале становится меньше 0.01 (≈ 0.036 в исходной). Итак, ошибка перестаёт уменьшаться начиная с p = 4; с ростом p сильно падает лишь число эпох: 2542 → 174 → 10.

Причина — в структуре ряда. Каждая гармоника cos(b·k) удовлетворяет линейному рекуррентному соотношению 2-го порядка, а сумма двух гармоник с разными частотами — соотношению 4-го порядка:

$$x(k) = c_1 \cdot x(k-1) - c_2 \cdot x(k-2) + c_1 \cdot x(k-3) - x(k-4)$$

$$c_1 = 2(\cos b + \cos d), \quad c_2 = 2 + 4 \cdot \cos b \cdot \cos d$$

$$x(k) = 3.514 \cdot x(k-1) - 5.060 \cdot x(k-2) + 3.514 \cdot x(k-3) - x(k-4)$$

Программная проверка даёт невязку $5 \cdot 10^{-14}$, т.е. соотношение точное. Значит, четырёх предыдущих значений достаточно, чтобы однозначно определить следующее. По двум значениям этого сделать нельзя: одной и той же паре (x(k−2), x(k−1)) в разные моменты соответствуют разные x(k), так как вторая гармоника находится в другой фазе. Неустранимая ошибка остаётся при любом обучении; даже лучшая линейная модель по двум точкам даёт RMSE = 0.121.

Большое окно (p = 12) не добавляет информации сверх p = 4: лишние входы линейно выражаются через первые четыре. Выигрыш есть только в скорости (10 эпох), но растут разброс результатов (std MAE 0.0042 против 0.0005), число весов, длина «разгона» прогноза (нужно 12 известных значений) и уменьшается число обучающих примеров (128 против 136). Немного меньший средний MAE при p = 12 объясняется тем, что сеть пересекает порог за одну эпоху и останавливается при $E_s \approx 6 \cdot 10^{-5} < E_e$, т.е. фактически обучается чуть точнее.

2. Как число скрытых нейронов N влияет на точность и скорость обучения? Наблюдается ли переобучение (малая ошибка на обучении при заметно большей на тесте)? Достаточно ли минимального N, чтобы воспроизвести сумму двух гармоник?

При p = 6 число нейронов почти не влияет ни на скорость, ни на точность: N = 1 — 186 ± 4 эпох, MAE 0.0273; N = 2 — 190 ± 16, 0.0274; N = 4 — 174 ± 12, 0.0275; N = 8 — 180 ± 7, 0.0255. Все 20 запусков достигли E_e. Различия лежат в пределах разброса между seed, лишь N = 8 даёт немного меньший MAE.

Переобучения нет: во всех конфигурациях ошибка на тесте не больше, чем на обучении (базовая сеть: 0.0276 на тесте и 0.0286 на обучении; для N = 8: 0.0255 и 0.0269).

Минимального N = 1 достаточно. Зависимость x(k) от предыдущих значений линейная (соотношение 4-го порядка, см. вопрос 1), а один сигмоидный

нейрон в рабочей области почти линеен: $\sigma(s) \approx 0.5 + s/4$. Линейный выходной нейрон масштабирует его выход обратно, и сеть p-1-1 воспроизводит линейный предиктор с точностью, достаточной для $Ee = 10^{-4}$. Для нелинейного ряда понадобилось бы больше нейронов.

3. Почему на выходе для задачи прогнозирования используется линейная функция, а не сигмоида (как в ЛР №3)? Что произошло бы при сигмоиде на выходе и при обучении на ненормализованных значениях?

Прогнозирование — задача регрессии: выход должен принимать любое вещественное значение, включая значения вне обучающего диапазона. Линейный выход $f(s) = s$ не ограничен и не насыщается ($f' = 1$), а градиент MSE для него простой: $\delta = d - y$. В ЛР №3 решалась задача классификации, где выход трактуется как вероятность, поэтому там уместна сигмоида.

С сигмоидой на выходе прогноз был бы ограничен интервалом (0; 1). Тестовые значения на нашем ряду выходят за него (−0.003 и 1.004 в нормализованной шкале), и сеть не смогла бы их воспроизвести. Кроме того, у краёв диапазона (экстремумы ряда) сигмоида насыщается: производная $\sigma' \to 0$ гасит градиент MSE (как в ЛР №2), и пики воспроизводились бы медленно и с систематической ошибкой.

При обучении на ненормализованных значениях ($x \in [-1.8; 1.8]$) сигмоидный выход не смог бы выдать отрицательные значения вообще. С линейным выходом обучение формально возможно, но масштаб входов и целей влияет на величину шагов: при том же η обучение может стать неустойчивым, сигмоиды скрытого слоя чаще насыщаются, а порог $Ee = 10^{-4}$ будет означать другую точность. Нормализация приводит все величины к единому масштабу и делает выбор η и Ee осмысленным.

4. Почему нормализация должна опираться только на обучающий участок? К чему приводит выход тестовых значений за диапазон [0; 1] и как это проявляется на ваших данных?

Тестовый участок моделирует будущее, неизвестное в момент обучения. Если брать min и max по всему ряду, в модель попадает информация из теста (утечка данных), и оценка качества становится завышенной. В реальной эксплуатации такой нормализации просто не будет. Поэтому параметры нормализации, как и веса, определяются только по обучающим данным, а к новым данным применяются в готовом виде.

На нашем ряду min и max обучающего участка равны −1.7840 и 1.7830, а на тесте есть значения −1.7958 и 1.7989. В нормализованной шкале тест занимает [−0.0033; 1.0045], т.е. немного выходит за [0; 1]. Сеть с линейным выходом экстраполирует такие значения без проблем: пик $x(157) = 1.7989$ предсказан

как 1.7986, а тестовый MAE (0.0276) не хуже обучающего (0.0286). С сигмоидой на выходе эти точки были бы недостижимы.

Сильный выход за диапазон опасен: сеть никогда не видела таких входов, и её ответ — неконтролируемая экстраполяция. В режиме функционирования окно с выбросом 3.0 (нормализованное значение 1.34) дало прогноз −3.70 при истинном значении −1.38: ошибка 2.3, т.е. 65 % размаха ряда.

5. Насколько устойчиво обучение к случайной инициализации весов (по сериям из 5 запусков)? Были ли запуски, не достигшие Ee за 20 000 эпох, и от чего это зависит (η, N, p)?

Обучение очень устойчиво. Во всех конфигурациях с $p \geq 4$ сошлись 5 из 5 запусков при небольшом разбросе: базовая сеть — 174 ± 12 эпох (162-194), при разных N — std 4-16 эпох, при $p = 4$ — 2542 ± 203. Разброс тестового MAE не превышает ±0.0015, кроме $p = 12$ (±0.0042).

Ee не достигли все 5 запусков с $p = 2$. Это определяется размером окна, а не инициализацией: при $p = 2$ точное решение не существует, и ошибка ограничена снизу $\approx 10^{-3}$. Скорость обучения в диапазоне 0.01-0.3 и число нейронов 1-8 на сходимость не влияют: все 5 запусков сходятся при любом из проверенных значений, меняется лишь число эпох ($\eta = 0.01$ — 4160, $\eta = 0.3$ — 174). Ещё одна причина недостижения Ee — шум на входах: при $\sigma = 0.02$ критерий не достигнут ни разу.

6. Как растёт ошибка рекурсивного многошагового прогноза с увеличением горизонта? Объясните накопление ошибки и сравните одношаговый прогноз с многошаговым. До какого горизонта прогноз можно считать пригодным?

На горизонте 30 шагов (сеть $p = 12$, $N = 4$) средняя ошибка рекурсивного прогноза 0.0336 против 0.0212 у одношагового на тех же точках, т.е. в 1.6 раза больше. Максимальная ошибка 0.086 на 13-м шаге не превышает порога 5 % размаха ряда ($0.05 \cdot 3.5947 = 0.180$). Ошибка растёт не монотонно, а волнообразно: она больше вблизи экстремумов и крутых участков ряда (шаги 11-15, 20-21, 30) и меньше около пологих участков. При продолжении рекурсии до конца тестового участка (60 шагов) MAE вырастает до 0.0666, максимум до 0.240, и порог 5 % впервые превышен на 51-м шаге.

Накопление ошибки: при одношаговом прогнозе все p входов — истинные значения, поэтому ошибка каждого шага независима и мала. При рекурсивном прогнозе предсказание с ошибкой становится входом следующего шага, и через p шагов окно состоит только из прогнозов. Ошибки входов проходят через сеть и складываются с её собственной ошибкой. Для нашего ряда (корни

характеристического уравнения лежат на единичной окружности) возмущения не затухают, а накапливаются в виде сдвига фазы и амплитуды гармоник.

По критерию 5 % размаха прогноз пригоден примерно до 50 шагов; с запасом (ошибка не больше ≈ 2.5 % размаха) — до 30 шагов, т.е. на весь требуемый горизонт.

7. Как согласуется различие между ошибкой на обучающем и тестовом участках с понятием обобщающей способности? Какие меры (ранняя остановка, уменьшение N, добавление шума, увеличение обучающей выборки) могли бы улучшить обобщение? Что показал эксперимент с шумом?

Обобщающая способность — это способность сети давать на новых данных такую же точность, как на обучающих. У нас тестовый MAE (0.0276) даже чуть меньше обучающего (0.0286), RMSE 0.0338 и 0.0342. Обобщение отличное: сеть выучила закон ряда (рекуррентное соотношение), а не отдельные точки. Тест порождён тем же детерминированным процессом без шума, поэтому переобучаться не на чем.

Если бы тестовая ошибка заметно превышала обучающую, помогли бы: ранняя остановка по ошибке на отдельной валидационной части (здесь её роль играет порог Ee); уменьшение числа нейронов и весов; добавление шума ко входам (регуляризация); увеличение обучающей выборки. Для рядов также важно, чтобы обучающий участок охватывал все характерные режимы.

Эксперимент с шумом (гауссов шум σ на входах при обучении, 5 запусков): σ = 0.005 — 191 эпоха, тестовый MAE 0.0282; σ = 0.01 — 3180 ± 799 эпох, MAE 0.0277; σ = 0.02 — Ee не достигнута ни разу, MAE 0.0764. На чистом детерминированном ряду шум не улучшает обобщение, поскольку улучшать нечего. Он лишь создаёт «пол» ошибки: при σ = 0.02 этот пол выше Ee, и обучение не может остановиться по критерию. Шум как регуляризация полезен на реальных зашумлённых данных и в избыточно больших сетях.

Вывод: Многослойный персептрон p-N-1 с сигмоидным скрытым слоем и линейным выходом, обученный онлайн-backprop на нормализованном по обучающему участку ряде $x(k) = 1.2\cdot\cos(0.28k) + 0.6\cdot\sin(0.65k)$, решает задачу прогнозирования. Базовая сеть 6-4-1 ($\eta = 0.3$) обучилась за 194 эпохи, одношаговый прогноз на тесте: MAE = 0.028, RMSE = 0.034, sMAPE = 10 %; ошибки на обучении и тесте совпадают — переобучения нет. Решающий параметр — размер окна: при p = 2 прогноз принципиально неточен (MAE 0.097, критерий не достигнут), начиная с p = 4 ошибка выходит на уровень, заданный порогом Ee. Это объясняется тем, что сумма двух гармоник —

линейный процесс 4-го порядка. Число скрытых нейронов (1-8) на результат почти не влияет. Обучение устойчиво к инициализации (все запуски с $p \geq 4$ сошлись). Рекурсивный прогноз на 30 шагов ($p = 12$, $N = 4$) держит ошибку ниже 5 % размаха ряда (максимум 0.086), а порог 5 % превышается только на 51-м шаге: ошибки накапливаются, но медленно. Шум на входах на чистом ряду обобщение не улучшает, а при $\sigma = 0.02$ делает критерий Ее недостижимым.